

INTERN PROJECT BRIEF

Helpdesk / Ticketing System

A beginner-friendly Node.js backend project

Stack	Node.js · Express · MongoDB · REST API
Level	Beginner — suitable after tutorials and 1–2 small projects
Expected Duration	3 – 4 weeks
Deliverable	Working REST API with Postman collection for testing

1. What You Are Building

You will build the backend (server-side) of a Helpdesk Ticketing System. Think of it like a simplified version of tools such as Freshdesk or Zendesk. When a customer has a problem, they create a ticket. An agent picks it up, works on it, and resolves it. A manager (admin) can see everything and get reports.

You do NOT need to build a frontend (no HTML pages, no CSS). Everything will be tested using Postman — a tool that lets you send requests to your API and see the responses.

2. Key Words You Must Know

Before you start coding, make sure you understand what each of these words means:

Term	Simple Meaning	Example
API	A set of URLs your server exposes so others can talk to it	POST /tickets creates a new ticket
REST	A style of building APIs using HTTP methods like GET, POST, PUT, DELETE	GET /tickets/5 returns ticket #5
Endpoint	One specific URL in your API	/tickets, /users/login
Middleware	Code that runs before your main function — for checking auth, logging, etc.	Check if user is logged in before showing ticket
JWT	A small encrypted token sent in every request to prove who you are	eyJhbGciOi... in the Authorization header
RBAC	Role-Based Access Control — different users can do different things	Customer can't delete tickets; Admin can
Schema	The shape/structure of data in your database	A ticket has title, status, createdBy fields

SLA	Service Level Agreement — a promise to respond/resolve within X hours	Priority 1 tickets must be resolved in 4 hours
Status	The current state of a ticket in its lifecycle	Open → In Progress → Resolved → Closed
Populate	Mongoose method to fill in referenced data automatically	Show agent's name instead of just their ID

3. User Roles

There are three types of users in this system. Each role has different permissions — this is called Role-Based Access Control (RBAC).

Role	Who Is This?	What Can They Do?
Customer	Person who has a problem and needs help	Create tickets, attach files, add comments on their own tickets, view status of their tickets
Agent	Support staff who solves tickets	View all tickets, change ticket status, add internal notes (hidden from customer), assign ticket to themselves
Admin	Manager who oversees everything	Do everything an Agent can + manage user accounts, view analytics dashboard, configure SLA rules

4. Ticket Statuses

Every ticket is always in one of these 5 states. The status can only move in certain directions (called a state machine). A closed ticket cannot be reopened.

OPEN New ticket, nobody working on it yet	IN PROGRESS An agent is actively working on it	WAITING Agent waiting for customer reply/info
RESOLVED Agent fixed the issue, pending confirmation	CLOSED Fully done. No more changes allowed	

Allowed transitions — only these moves are valid:

From Status	Can Move To	Who Can Do It
OPEN	IN PROGRESS, CLOSED	Agent, Admin
IN PROGRESS	WAITING, RESOLVED, OPEN	Agent, Admin
WAITING	IN PROGRESS, CLOSED	Agent, Admin
RESOLVED	CLOSED, IN PROGRESS	Agent, Admin, Customer

CLOSED	Cannot change	Nobody
--------	---------------	--------

5. Modules to Build

Think of your project as 5 separate mini-apps that work together. Build them one by one in the order below.

Module 1 — Auth (Authentication)

This is the login/signup system. Every API request must prove who the user is by sending a JWT token.

- POST /auth/register — Register a new user with name, email, password, and role
 - Password must be hashed using bcryptjs before saving
 - role can be: customer, agent, admin
- POST /auth/login — Login with email + password, returns a JWT token
 - Token should expire in 7 days
- GET /auth/me — Returns the logged-in user's info (requires token)

Key concept: The JWT token is like a hall pass. Once logged in, the user sends this token with every request. Your middleware checks it and decides if the user is allowed in.

Module 2 — Tickets (Core Feature)

This is the main feature of the whole system. Most of your time will be spent here.

- POST /tickets — Customer creates a new ticket
 - Required fields: title, description, priority (low / medium / high / urgent)
 - Auto-set: status = open, createdBy = logged-in user, createdAt = now
- GET /tickets — Get list of tickets
 - Customer sees only their own tickets
 - Agent/Admin sees all tickets
 - Support filter by status, priority, assigned agent
- GET /tickets/:id — Get full details of one ticket
- PATCH /tickets/:id/status — Change ticket status (Agent/Admin only)
 - Validate the status transition is allowed (see Section 4)
- PATCH /tickets/:id/assign — Assign ticket to an agent (Agent/Admin only)
- DELETE /tickets/:id — Delete ticket (Admin only)

Module 3 — Comments

Comments are messages attached to a ticket. There are two types:

- Public Comment — Both the customer and agents can see it

- Internal Note — Only agents and admins can see it (hidden from customer)
- POST /tickets/:id/comments — Add a comment or internal note
 - isInternal: true makes it an internal note
 - Only Agent/Admin can post internal notes
- GET /tickets/:id/comments — Get all comments for a ticket
 - Customers only see public comments
 - Agents/Admins see all including internal notes

Module 4 — File Attachments

Customers should be able to upload files (screenshots, logs, etc.) when creating a ticket or adding a comment.

- Use the multer npm package for handling file uploads
- Limit file size to 5MB, allow only jpg, png, pdf, txt
- Store uploaded files in an /uploads folder on disk
- Save the file path in the ticket or comment document in MongoDB
- GET /tickets/:id/attachments — List all files attached to a ticket

You do not need cloud storage (S3, Cloudinary). Local disk storage is fine for this project.

Module 5 — Admin & Reports

The admin panel gives managers a bird's-eye view of how the support team is performing.

User Management

- GET /admin/users — List all users
- PATCH /admin/users/:id — Update user role or deactivate account
- DELETE /admin/users/:id — Delete a user

Analytics Endpoints

- GET /admin/analytics/overview — Returns summary numbers
 - Total tickets, open tickets, resolved today, avg resolution time
- GET /admin/analytics/by-status — Count of tickets grouped by status
- GET /admin/analytics/by-agent — Number of tickets assigned to each agent
- GET /admin/analytics/sla-breaches — List of tickets where SLA was missed

6. SLA Tracking (Advanced Feature)

SLA = Service Level Agreement. It is a rule that says tickets must be responded to / resolved within a certain time based on their priority.

Priority	First Response Due	Resolution Due
urgent	1 hour	4 hours
high	4 hours	8 hours
medium	8 hours	24 hours
low	24 hours	72 hours

What you need to build:

- When a ticket is created, calculate and store responseDeadline and resolutionDeadline based on its priority
- Each time you fetch a ticket, check if deadlines have passed and add an isSlaBreached: true/false field
- The GET /admin/analytics/sla-breaches endpoint should return all tickets where deadline passed and status is not resolved/closed

7. Auto-Assignment (Advanced Feature)

When a new ticket is created, automatically assign it to the agent who currently has the fewest open tickets.

- When POST /tickets is called, query the database to find all agents
- Count how many open tickets each agent has
- Assign the new ticket to the agent with the lowest count
- If no agents exist, leave assignedTo as null

8. Email Notifications (Advanced Feature)

Send emails when important things happen. Use nodemailer with a free test account from Ethereal Email (<https://ethereal.email>) — no real emails are sent, you can see them in the browser.

- New ticket created → Email the customer confirming receipt
- Ticket assigned → Email the assigned agent
- Status changed → Email the customer
- New public comment → Email the customer

Build a simple emailService.js helper file with a sendEmail(to, subject, body) function. Call it from your route handlers.

9. Database Schemas (MongoDB + Mongoose)

You will have 3 collections (tables) in MongoDB. Here is the exact shape of each document:

User Schema

name	String, required
email	String, required, unique
password	String, hashed with bcryptjs
role	Enum: customer agent admin — default: customer
isActive	Boolean — default: true
createdAt	Date — auto

Ticket Schema

title	String, required, max 200 chars
description	String, required
status	Enum: open in_progress waiting resolved closed — default: open
priority	Enum: low medium high urgent — default: medium
createdBy	ObjectId ref → User (the customer)
assignedTo	ObjectId ref → User (the agent) — can be null
attachments	Array of strings (file paths)
tags	Array of strings — optional
responseDeadline	Date — calculated from priority on create
resolutionDeadline	Date — calculated from priority on create
resolvedAt	Date — set when status becomes resolved
createdAt / updatedAt	Date — auto (use timestamps: true)

Comment Schema

ticket	ObjectId ref → Ticket
author	ObjectId ref → User
body	String, required
isInternal	Boolean — default: false
attachments	Array of strings (file paths)
createdAt	Date — auto

10. Recommended Folder Structure

Organise your project like this. Each folder has one job:

Path	What Goes Here
helpdesk/	Root of your project
server.js	Entry point — starts Express server
.env	Secret config: DB URL, JWT secret, port
/config/db.js	MongoDB connection code
/models/User.js	Mongoose schema for users
/models/Ticket.js	Mongoose schema for tickets
/models/Comment.js	Mongoose schema for comments
/routes/auth.js	Login / register endpoints
/routes/tickets.js	All ticket CRUD endpoints
/routes/comments.js	Comment endpoints
/routes/admin.js	Admin-only endpoints
/middleware/auth.js	JWT verification middleware
/middleware/rbac.js	Role permission checking
/services/slaService.js	SLA deadline calculation logic
/services/emailService.js	Email sending helper
/uploads/	Uploaded files stored here

11. npm Packages to Use

Install all required packages using npm install. Here is what each one does:

Package Name	What It Does	When You Use It
express	Web framework — handles routes, requests, responses	Always — your whole server runs on this
mongoose	Connects Node.js to MongoDB, lets you define schemas	Every database operation
bcryptjs	Hashes (scrambles) passwords so they can't be read	Register and login
jsonwebtoken	Creates and verifies JWT tokens	Login and every protected route
multer	Handles file uploads from the request body	Attaching files to tickets
nodemailer	Sends emails from your Node app	Notifications module
dotenv	Loads secret values from a .env file	Always — for DB URL, JWT secret
express-validator	Validates incoming request data (email format, required fields)	All routes that accept input

morgan	Logs every request to the console — helpful for debugging	Development only
cors	Allows frontend apps on other domains to call your API	Always — needed for Postman/browser

12. What You Will Submit

Code Deliverables
• Full GitHub repository with all code committed • README.md with: how to run the project, environment variables needed, brief description of each module • Postman collection (.json file) with all API endpoints ready to test • .env.example file showing all required env vars (without real values)

Working Features (Minimum)
• User registration and login with JWT • CRUD for tickets with role-based access • Status transitions with validation • Public comments and internal notes • File upload for attachments • Admin user management and basic analytics

Bonus Features (Nice to Have)
• SLA tracking with deadline calculation • Auto-assignment to least-busy agent • Email notifications via Nodemailer • Pagination on list endpoints (page and limit query params)

13. Implementation Approach

Build the modules in the order below. Each step builds on the previous one — do not skip ahead. Move to the next step only when the current one is fully working in Postman.

Step	What to Build	Done When...
Step 1	Project setup + Auth — server.js, DB connection, User model, register & login endpoints	You can register, login, and get /auth/me working in Postman

Step 2	Tickets CRUD — Ticket model and all 6 ticket endpoints with role-based access	All ticket endpoints work with correct role checks
Step 3	Comments, File Uploads, Admin — Modules 3, 4, and 5 in order	Comments, uploads, and admin analytics all work
Step 4	Advanced features + Cleanup — SLA, Auto-assign, Emails, Postman collection, README	Postman collection complete, README written, repo clean

14. Rules & Expectations

You Must Follow These Rules

- Never store plain-text passwords — always use bcryptjs to hash
- Every route that needs login must check the JWT token first
- Customers must never see another customer's tickets
- Internal notes must never be visible to customers
- Status transitions must be validated — no jumping to invalid states
- All inputs must be validated — no empty titles, invalid emails, etc.
- Return proper HTTP status codes: 200, 201, 400, 401, 403, 404, 500

Good Practices (You Will Be Reviewed On These)

- Use async/await with try/catch for all database operations
- Write clear error messages that actually explain what went wrong
- Use meaningful variable and function names — no x, temp, abc
- Comment your code where the logic is non-obvious
- Commit to Git regularly — at least once per working session

15. Quick Reference — HTTP Status Codes

Code	Meaning	When to Use
200 OK	Request succeeded	GET, successful PATCH/DELETE
201 Created	New resource created	POST that creates something
400 Bad Request	User sent bad data	Missing fields, wrong format
401 Unauthorized	Not logged in / invalid token	No token or expired token
403 Forbidden	Logged in but not allowed	Customer trying admin route
404 Not Found	Resource doesn't exist	Ticket ID not found in DB

500 Server Error	Something broke on your side	Unhandled exception in code
------------------	------------------------------	-----------------------------

16. Tips for Success

1. Start with Module 1 (Auth) and make sure login and token verification work before building anything else. Everything else depends on it.
2. Test every endpoint in Postman right after you write it — don't wait until the end.
3. When stuck, read the error message carefully. Most Node/Mongoose errors tell you exactly what went wrong.
4. Google is your friend. Searching 'mongoose find by field' or 'express middleware next()' is normal and expected.
5. Commit your code to GitHub every time something new works — not at the end of the project.
6. Ask questions early. If you are stuck for more than 30 minutes, reach out — don't waste half a day on one bug.

Good luck! Build it step by step and you will get there.